



Universidad Nacional Autónoma de México

Facultad de Ciencias

Teoría de la Computación | Grupo: 7084

Tarea 1

Frida Thais Espinosa Romero

Ariel Pérez Cancino

Erika Renata Reyes Núñez



1.- Sean Σ un alfabeto, $w \in \Sigma^*$ y $n \in \mathbb{N}$. Prueba lo siguiente:

a) $(w^n)^R = (w^R)^n$

· Concatenación: $(uv)w = u(vw)$

Caso base ($n = 0$)

· $v\lambda = \lambda v = v$

Usando def potencia, $(w^n)^R \Rightarrow (w^0)^R = \lambda$

$(\lambda)^R = \lambda$ ✓

· Reverso (w^R)

$(w^R)^n \Rightarrow (w^0)^n = (\lambda)^0 = \lambda$ ✓

$(uv)^R = v^R u^R$

∴ Se cumple para $n = 0$

· Potencia (w^n) :

$w^0 = \lambda$

$w^{n+1} = w^n w$

H. I.

Supongamos que se cumple para cualquier

$k \in \mathbb{N}$

$(w^k)^R = (w^R)^k$

◦ Def recursiva de Potencia

$w^{k+1} = w^k w$

Paso Inductivo

Sea $n = k + 1$

P.D. $(w^{k+1})^R = (w^R)^{k+1}$

Longitud de la cadena

vacía: $|\lambda| = 0$

Usando la def recursiva de potencia, sabemos que $w^{k+1} = w^k w$, entonces:

$(w^{k+1})^R = (w^k w)^R$

$\stackrel{\text{Reversa}}{=} w^R (w^k)^R$

$\stackrel{\text{H.I}}{=} w^R (w^R)^k$

$\stackrel{\text{Potencia}}{=} (w^R)^{k+1}$

∴ $(w^n)^R = (w^R)^n$

* Propiedad de la longitud

sobre la concatenación: $|u \cdot v| = |u| + |v|$

b) $|w^n| = n \cdot |w|$

Caso base $n = 0$

$w^n \Rightarrow |w^0| = |\lambda| = 0$

$n \cdot |w| = 0 \cdot |w| = |\lambda| = 0$

∴ Se cumple para $n = 0$

H. I.

Supongamos que se cumple para cualquier $k \in \mathbb{N}$

$$|w^k| = k \cdot |w|$$

Paso Inductivo

Sea $k = k + 1$

$$|w^{k+1}| = k + 1 \cdot |w|$$

$$\begin{aligned} |w^{k+1}| &= |w^k w| \\ &\stackrel{*}{=} |w^k| + |w| \\ &\stackrel{\text{H.I}}{=} k \cdot |w| + |w| \\ &\stackrel{\circ}{=} k \cdot |w| \end{aligned}$$

$\therefore$ Se cumple que $|w^n| = n \cdot |w|$

2.- Utilizaremos doble contención:

$\subseteq$

P.D para toda cadena $w \in L$, se cumple $w = w^R$

Utilizando Inducción:

-Caso Base:

■ $w = \lambda$:

Por definición recursiva de la reversa, sabemos que $\lambda^R = \lambda$

Por lo tanto se cumple

$$\lambda = \lambda^R$$

■ $(w = a \mid a \in \Sigma)$:

por definición recursiva de la reversa:

$$a^R = (\lambda a)^R = a \cdot \lambda^R = a \cdot \lambda = a$$

por lo tanto $a = a^R$

-Hipótesis de Inducción:

Supongamos que para toda cadena $w \in L$, se cumple:

$$w = w^R$$

-Paso Inductivo:

P.D que:

$$(awa)^R = awa$$

por la propiedad de la reversa de una concatenación $(\alpha\beta\gamma)^R = \gamma^R\beta^R\alpha^R$ obtenemos:

$$((awa)^R = (a^R) \cdot (w^R) \cdot (a^R))$$

Aplicando el caso base ($a^R = a$):

$$a^R \cdot w^R \cdot a^R = a \cdot w^R \cdot a$$

Aplicando la hipótesis de inducción ($w^R = w$):

$$a \cdot w \cdot a = awa$$

Por lo tanto

$$(awa)^R = awa$$

Se cumple que

$$L \subseteq \{w \in \Sigma^* \mid w = w^R\}$$

$\supseteq$

P.D que $\{w \in \Sigma^* \mid w = w^R\} \subseteq L$

Si $w \in \Sigma^*$ es palindromo (es decir $w = w^R$), entonces $w \in L$.

Haremos inducción sobre la longitud de w ($|w| = n$), ya que el conjunto de palíndromos está definida por propiedades (como la longitud de la cadena) y no por una definición recursiva.

-Caso Base:

■ $|w| = 0$:

Si $|w| = 0$ la unica cadena que cuumple con esta propiedad es

$$w = \lambda$$

■ $|w| = \lambda$:

Por definción de L , sabemos que $\lambda \in L$

■ $|w| = 1$:

Si $|w| = 1$, entonces

$$w = a$$

para $a \in \Sigma$.

Por definción de L , sabemos que $a \in L$

-Hipótesis de Inducción:

Supongamos que para toda cadena $w \in \Sigma^*$ con longitud: $0 \leq |w| \leq n$, con $n \geq 2$, si w es un palíndromo ($w = w^R$), entonces $w \in L$.

-Paso Inductivo:

Sea $w \in \Sigma^*$ con $|w| = n \geq 2$ tal que $w = w^R$, w es un palíndromo

P.D que $w \in L$

Como $|w| \geq 2$, podemos escribir:

$$w = \alpha \cdot u \cdot \beta$$

donde $\alpha, \beta \in \Sigma$ y $u \in \Sigma^*$

Además $|w| = |\alpha| + |u| + |\beta|$, es decir:

$$|w| = 1 + |u| + 1$$

$$|w| = |u| + 2$$

$$|u| = |w| - 2$$

$$|u| = n - 2$$

Aplicando ($w = w^R$) obtenemos:

$$\begin{aligned}\alpha \cdot u \cdot \beta &= (\alpha \cdot u \cdot \beta)^R = \beta^R \cdot u^R \cdot \alpha^R \\ &= \beta \cdot u^R \cdot \alpha\end{aligned}$$

Igualando los extremos de la cadena obtenemos:

$$\alpha = \beta$$

$$u = u^R$$

Podemos deducir que:

- w es de la forma $w = \alpha \cdot u \cdot \alpha$
- u es un palíndromo, ya que $u = u^R$ y $|u| = n - 2 < n$

Por hipótesis de inducción, $u \in L$ Como $u \in L$ y $\alpha \in \Sigma$, por el tercer punto de la definición de L :

$$\alpha \cdot u \cdot \alpha \in L$$

Es decir:

$$w \in L$$

Al haber demostrado ambas contenciones, concluimos que:

$$L = \{w \in \Sigma^* \mid w = w^R\}$$

3.- Sea $L \subseteq \{a, b\}^*$ el lenguaje definido de forma inductiva por:

- $\lambda \in L$
- Si $w \in L$, entonces $wa \in L$ y $wba \in L$

a) Prueba que para cualquier cadena $u \in L$, u tiene mayor o igual cantidad de a 's que de b 's.

Demostración por inducción estructural sobre L Sea $N_a(u)$ el número de símbolos a en la cadena u y sea $N_b(u)$ el número de símbolos b en la cadena u . Demostremos que $N_a(u) \geq N_b(u)$.

Caso base: Si $u = \lambda$, entonces $N_a(\lambda) = 0$ y $N_b(\lambda) = 0$. Por lo tanto, se cumple trivialmente que $N_a(\lambda) \geq N_b(\lambda)$.

Hipótesis de inducción: Supongamos que la propiedad se cumple para alguna cadena $w \in L$, es decir, que $N_a(w) \geq N_b(w)$.

Paso inductivo: Evaluamos las reglas de formación:

- **Caso 1: Sea $u = wa$**

Si agregamos una a , obtenemos que:

$$\begin{aligned} N_a(wa) &= N_a(w) + 1 \\ N_b(wa) &= N_b(w) \end{aligned}$$

Por nuestra hipótesis de inducción sabemos que $N_a(w) \geq N_b(w)$. Al sumar 1 a $N_a(w)$, obtenemos $N_a(w) + 1 > N_b(w)$, por lo que se sigue cumpliendo que $N_a(wa) \geq N_b(wa)$.

- **Caso 2: Sea $u = wba$**

Si agregamos ahora ba , obtenemos que:

$$\begin{aligned} N_a(wba) &= N_a(w) + 1 \\ N_b(wba) &= N_b(w) + 1 \end{aligned}$$

Por nuestra hipótesis de inducción, $N_a(w) \geq N_b(w)$. Si sumamos 1 a ambos lados de la desigualdad, se mantiene el orden:

$$N_a(w) + 1 \geq N_b(w) + 1$$

Esto significa que se sigue cumpliendo que $N_a(wba) \geq N_b(wba)$.

Queda demostrado por inducción estructural que $N_a(u) \geq N_b(u)$ para toda $u \in L$.

b) Prueba que para cualquier cadena $u \in L$, u no tiene como subcadena bb .

Primero determinemos un lema auxiliar que nos ayudara: *Ninguna cadena $w \in L$ (distinta de λ) termina en b .*

Demostración del lema Por inducción estructural:

- **Caso base:** λ no termina en b (carece de elementos).
- **Paso inductivo:** Por las reglas que nos brinda el problema, se generan cadenas wa y wba . Ambas reglas agregan siempre una a al final de la nueva cadena, por lo que nunca sucederá que una cadena termine en b .

Ahora probemos que cualquier cadena $u \in L$ no tiene como subcadena bb .

Demostración por inducción estructural

Caso base: Para $u = \lambda$. Como la cadena vacía carece de símbolos, es imposible que contenga la subcadena bb .

Hipótesis de inducción: Supongamos que para alguna cadena $w \in L$, w no contiene la subcadena bb .

Paso inductivo: Evaluamos las reglas de formación:

- **Caso 1: Para la cadena wa .**

Si concatenamos una a al final de alguna cadena w , no se formará una subcadena bb en la frontera puesto que el nuevo símbolo es una a . Como, por hipótesis de inducción, w no contiene a la subcadena bb , entonces la nueva cadena wa tampoco la contendrá.

- **Caso 2: Para la cadena wba .**

Si concatenamos los símbolos ba a alguna cadena w , debemos analizar la frontera. Por el lema demostrado al principio, sabemos que ninguna $w \in L$ termina en b . Dado que w no termina en b , al concatenarle ba es imposible que se forme la subcadena bb . Además, por hipótesis de inducción, sabemos que w internamente no contiene subcadenas bb , y el sufijo ba tampoco la contiene. Por lo tanto, la nueva cadena wba estará libre de la subcadena bb .

Dado que ambos casos se cumplen, demostramos por inducción estructural que ninguna cadena $u \in L$ contiene la subcadena bb .

4.- Utilizando doble contención

$\subseteq$ |

P.D que $L \subseteq L^+$

Por definición de cerradura inductiva dada en clase:

$$L^+ = \bigcup_{n \geq 1} L^n = L^1 \cup L^2 \cup \dots$$

Para $n = 1$ aplicando la definición de potencia: $L^1 = L \cdot L^0 = L \cdot \{\lambda\} = L$

Dado que $L^1 = L$, se cumple trivialmente que

$$L \subseteq L^+$$

$\supseteq$ |

Como $L^+ = \bigcup_{n \geq 1} L^n$ para demostrar que $L^+ \subseteq L$ hacemos inducción sobre $n \geq 1$.

-Caso Base: ($n = 1$):

$$L^1 = L \cdot L^0 \stackrel{\text{def}}{=} L\{\lambda\} = L$$

Dado que $L^1 = L$, se cumple trivialmente que

$$L \subseteq L$$

-Hipótesis de Inducción: Supongamos que se cumple para k , es decir:

$$L^k \subseteq L$$

-Paso Inductivo: P.D que $L^{k+1} \subseteq L$

Por definición de potencia:

$$L^{k+1} = L^k \cdot L$$

Por Hipotesis de Inducción sabemos que:

$$L^k \subseteq L$$

Entonces:

$$L^{k+1} = L^k \cdot L \subseteq L \cdot L$$

Por H.I del problema ($L^2 \subseteq L$) Asi:

$$L^{k+1} = L^k \cdot L$$

$$L^k \cdot L \subseteq L^2 \subseteq L$$

Entonces:

$$L^{k+1} \subseteq L$$

Por lo tanto

$$L^+ = \bigcup_{n \geq 1} L^n \subseteq L$$

Al haber demostrado ambas contenciones, concluimos que:

$$L^+ = L$$

5.- Sea Σ un alfabeto y sean $L_1, L_2, L_3 \subset \Sigma^*$. Diga si:

a) $L_1(L_2 \cap L_3) = L_1L_2 \cap L_1L_3$

b) $(L_1L_2)^* = L_1^*L_2^*$

a) $L_1(L_2 \cap L_3) = L_1L_2 \cap L_1L_3$

Contraejemplo

Sean $L_1 = \{ab, abc\}$, $L_2 = \{cba\}$, $L_3 = \{ba\}$

$(L_2 \cap L_3) = \{cba\} \cap \{ba\} = \emptyset$

$L_1(L_2 \cap L_3) = \{ab, abc\}\emptyset = \emptyset$

$L_1L_2 = \{ab, abc\} \cdot \{cba\} = \{abcba, abccba\}$

$L_1L_3 = \{ab, abc\} \cdot \{ba\} = \{abba, abcba\}$

$L_1L_2 \cap L_1L_3 = \{abcba\}$

$\emptyset \neq \{abcba\}$

$\therefore L_1(L_2 \cap L_3) \neq L_1L_2 \cap L_1L_3$

Sabemos que
 $\mathcal{L}(\alpha) \cap \mathcal{L}(\beta) = \mathcal{L}(\alpha \cap \beta)$ y
 $\mathcal{L}(\alpha\beta) = \mathcal{L}(\alpha) \cdot \mathcal{L}(\beta)$

b) $(L_1L_2)^* = L_1^*L_2^*$

Contraejemplo

$L_1 = \{ab, abc\}$, $L_2 = \{cba\}$

$L_1L_2 = \{ab \cdot cba, abc \cdot cba\} = \{abcba, abccba\}$

$L_1L_2^2 = \{abcbaabcba, abcbaabccba, abccbaabcba, abccbaabccba\}$

$L_1^*L_2^* = \{ab, abc, ab, abc\}\{cba, cba\}$

$w = abcbabcba \in (L_1L_2)^*$

$\therefore (L_1L_2)^* \neq L_1^*L_2^*$

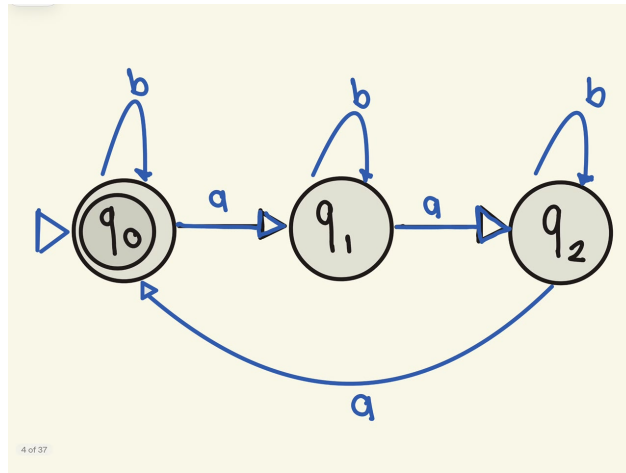
$\mathcal{L}(A^*) = (\mathcal{L}(A))^*$

$\mathcal{L}(AB) = \mathcal{L}(A)\mathcal{L}(B)$

6.- Describe un autómata que produzca el lenguaje $L = \{w \in \{a, b\}^* : |w|_a \equiv 0 \pmod{3}\}$, argumenta.

Definición formal del autómata:

El Autómata se define:



- **Estados (Q):** $\{q_0, q_1, q_2\}$
- **Alfabeto (Σ):** $\{a, b\}$
- **Estado inicial (q_0):** q_0
- **Estados de aceptación (F):** $\{q_0\}$

Semántica de los estados:

- q_0 : El número de letras a procesadas es múltiplo de 3 (residuo 0).
- q_1 : El número de letras a procesadas deja un residuo de 1 al dividir entre 3.
- q_2 : El número de letras a procesadas deja un residuo de 2 al dividir entre 3.

Argumento de correctitud:

El autómata funciona como un contador circular de tres posiciones (q_0, q_1, q_2) que registra el residuo de dividir la cantidad de letras a entre tres, ignorando por completo las letras b mediante bucles que lo mantienen en su estado actual. Al iniciar en q_0 (residuo cero), cada nueva a lo hace avanzar un paso, de modo que la primera lo lleva a q_1 (sobra una), la segunda a q_2 (sobran dos), y la tercera cierra el ciclo regresándolo a q_0 ; debido a que q_0 es su único estado de aceptación, el autómata garantiza que la cadena de texto solo sea aprobada si las letras a completan ciclos exactos de tres, asegurando que su cantidad sea un múltiplo de 3 sin importar cuántas b existan en la palabra.

7.-

Consideramos la gramática $G = \langle \Gamma, S, \Sigma, P \rangle$, definida como:

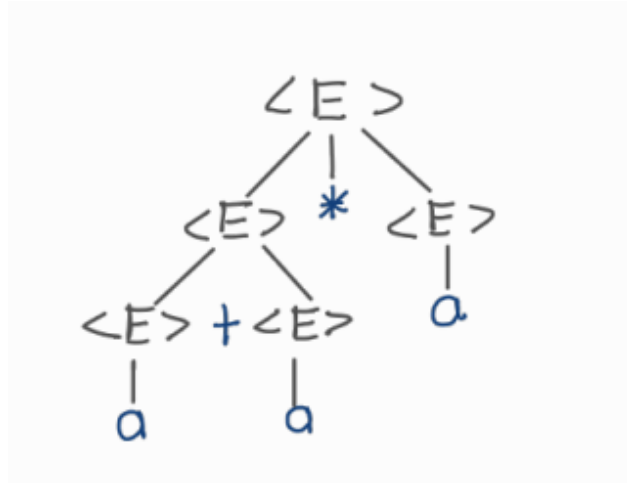
- $\Gamma = \{\langle E \rangle\}$
- $S = \langle E \rangle$
- $\Sigma = \{a, +, *\}$
- Reglas de producción P :

$$\langle E \rangle \rightarrow \langle E \rangle + \langle E \rangle \mid \langle E \rangle * \langle E \rangle \mid a$$

a) **Derivación 1:** Agrupamos la suma primero: $(a + a) * a$

Comenzamos aplicando la regla de la multiplicación en el nodo raíz y luego expandimos el subárbol izquierdo con la regla de la suma:

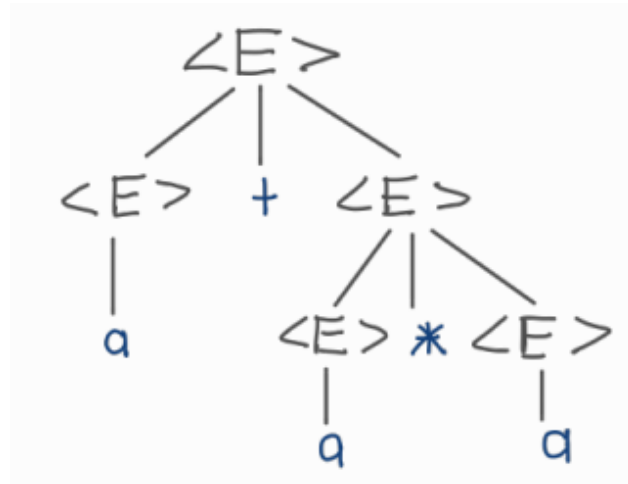
1. $\langle E \rangle \Rightarrow \langle E \rangle * \langle E \rangle$
2. $\langle E \rangle * \langle E \rangle \Rightarrow (\langle E \rangle + \langle E \rangle) * \langle E \rangle$
3. Sustituyendo los no terminales por el terminal a : $(a + a) * a \Rightarrow a + a * a$



Derivación 2: Agrupamos la multiplicación primero $a + (a * a)$:

Comenzamos aplicando la regla de la suma en el nodo raíz y luego expandimos el subárbol derecho con la regla de la multiplicación:

1. $\langle E \rangle \Rightarrow \langle E \rangle + \langle E \rangle$
2. $\langle E \rangle + \langle E \rangle \Rightarrow \langle E \rangle + (\langle E \rangle * \langle E \rangle)$
3. Sustituyendo los no terminales por el terminal a : $a + (a * a) \Rightarrow a + a * a$



b) Al existir dos árboles de derivación para la misma cadena $w = a + a * a$, podemos concluir que la gramática G es ambigua. Esto puede ser un problema en el compilador ya que el orden de evaluación de las operaciones deja de ser único. Al no ver una jerarquía clara puede hacer dos casos:

- Si se evalúa primero la suma, interpreta la cadena como $(a + a) * a$
- Si se evalúa primero la multiplicación, interpreta la cadena como $a + (a * a)$

Al poder modificar el orden en el que el compilador evalúa las operaciones de una misma expresión se pueden generar resultados distintos.

c)

Definimos la nueva gramática como $X = \langle \Gamma', S', \Sigma, R' \rangle$, donde:

- $\Gamma' = \{ \langle E \rangle, \langle T \rangle, \langle F \rangle \}$
- $S' = \langle E \rangle$
- $\Sigma = \{ a, +, * \}$
- Reglas de producción R' :

$$\begin{aligned} \langle E \rangle &\rightarrow \langle E \rangle + \langle T \rangle \mid \langle T \rangle \\ \langle T \rangle &\rightarrow \langle T \rangle * \langle F \rangle \mid \langle F \rangle \\ \langle F \rangle &\rightarrow a \end{aligned}$$

Para priorizar operaciones, separamos la gramática en niveles : la suma $+$ se queda arriba ($\langle E \rangle$) y la multiplicación $*$ se va más abajo ($\langle T \rangle$). Esto obliga al árbol a resolver siempre la multiplicación antes que la suma.

Al escribir $\langle E \rangle \rightarrow \langle E \rangle + \langle T \rangle$, aseguramos que las operaciones se vayan agrupando de izquierda a derecha sin confusiones.

Resultado: Para la cadena $a + a * a$, esta nueva gramática solo permite construir un único árbol posible, resolviendo así el problema de la ambigüedad.

8.- Escribe una gramática que genere un fragmento del español capaz de producir las oraciones “el gato duerme” y “el perro come la croqueta”, pero no “duerme el gato” ni “el croqueta come gato”. Identifica claramente terminales, no terminales, símbolo inicial y producciones.

Para evitar generar oraciones como “duerme el gato” ó “el croqueta come gato”, voy a generar una serie de reglas:

1. Todas las oraciones deben empezar con un artículo
2. Deben de concordar el género y el pronombre así se evita “el croqueta”
3. Después de un sustantivo, debe de ir un verbo, cumpliendo así la siguiente estructura:
pronombre + sustantivo + verbo
4. Pueden haber dos tipos de verbo
 - Verbo seguido del complemento
 - Verbo sin complemento

$G = \langle V, T, S, P \rangle$

$T = \langle \text{el, la, gato, perro, croqueta, duerme, come} \rangle$

$V = \{S, \text{Sujeto}_m, \text{Sujeto}_f, A_M, A_F, M, F, V_c, V\}$

S : Símbolo inicial

Suj_m : Sujeto masculino

Suj_f : Sujeto femenino

A_M : Artículo masculino

A_F : Artículo femenino

M : Sustantivo Masculino

F : Sustantivo Femenino

V_c : Verbo con complemento

V : Verbo sin complemento

Reglas de producción

$S \rightarrow \text{Suj}_m V \mid \text{Suj}_m V_c \text{Suj}_f$

$\text{Suj}_m \rightarrow A_M M$

$A_M \rightarrow \text{el}$

$A_F \rightarrow \text{la}$

$M \rightarrow \text{gato} \mid \text{perro}$

$F \rightarrow \text{croqueta}$

$V \rightarrow \text{duerme}$

$V_c \rightarrow \text{come}$

“el gato duerme”

$S \rightarrow \text{Suj}_m V \rightarrow A_M MV \rightarrow \text{el } MV$
 $\rightarrow \text{el gato } V \rightarrow \text{el gato duerme}$

“el perro come la croqueta”

$S \rightarrow \text{Suj}_m V_c \text{Suj}_f \rightarrow A_M MV_c \text{Suj}_f$
 $\rightarrow \text{el perro come } \text{Suj}_f \rightarrow \text{el perro come } A_F F$
 $\rightarrow \text{el perro come la } F \rightarrow \text{el perro come la croqueta}$